IDLCV Project IV: Object Detection
Group 47

Atai Mozhdah s224538
Carrara Alessandra s252976
Lykouresis Nikolaos Dimitrios s243174
Rueda Alberto s253057
Tsilingeridis Emmanouil s253154

1. Introduction

Potholes are structural failures in road surfaces, typically caused by water accumulation and traffic stress, which pose significant safety risks to vehicles and pedestrians. As road infrastructure ages, the need for automated inspection systems becomes essential for timely maintenance and improved traffic safety. However, detecting potholes in "in-the-wild" environments is a challenging computer vision task due to variations in lighting, road texture, and the visual similarity between potholes, shadows, and patches.
The objective of this project is to construct a complete deep learning object detection pipeline capable of identifying potholes under these realistic conditions. The methodology is implemented in three distinct stages: (1) generating unsupervised region proposals using Selective Search, (2) training a Convolutional Neural Network (CNN) to classify these proposals, and (3) applying Non-Maximum Suppression (NMS) to refine the final detections. We evaluate the system using the Potholes dataset, utilizing the Average Precision (AP) metric to quantify performance.

2. Part 1 - Object Proposals

The goal of this part is to extract and evaluate region proposals that may contain potholes. We applied **Selective Search** to generate candidate bounding boxes and visualized them alongside the ground-truth annotations from the dataset.

2.1. Region Proposal Extraction

Selective Search was applied to each image to generate region proposals. An example output is shown in Figure 1. Only a subset of proposals is visualized due to the large number of candidate boxes.

2.2. Evaluation Using IoU

To evaluate the quality of the proposals, we computed the **Intersection over Union (IoU)** between each proposal and the ground-truth boxes. A histogram of IoU log-y values is shown in Figure 2, illustrating that only a small number of proposals achieve a high overlap.

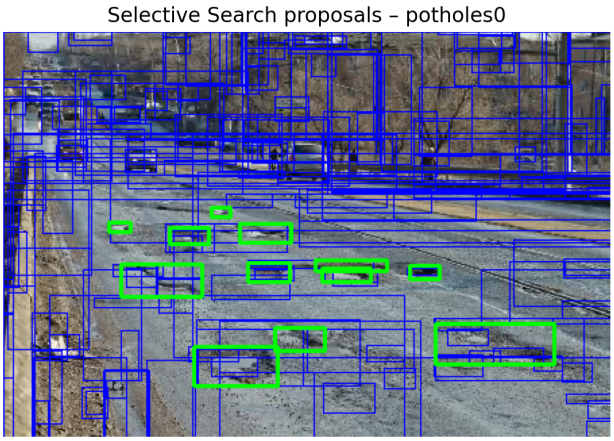

Figure 1. Example of region proposals generated using Selective Search.

2.3. Observation

The distribution shows a large number of low-IoU proposals, meaning most regions do not correspond to potholes. However, Selective Search ensures high recall, which is crucial for the next stage training a classifier to distinguish between potholes and background.

3. Part 2 - Object Detector

3.1. CNN for proposal classification

A compact convolutional neural network (`SimpleCNN`) was implemented to classify each cropped proposal into either *pothole* or *background*. The network takes as input an RGB patch resized to 128×128 (shape $3 \times 128 \times 128$). The architecture is composed of four convolutional blocks, each including:

- 3×3 convolution,

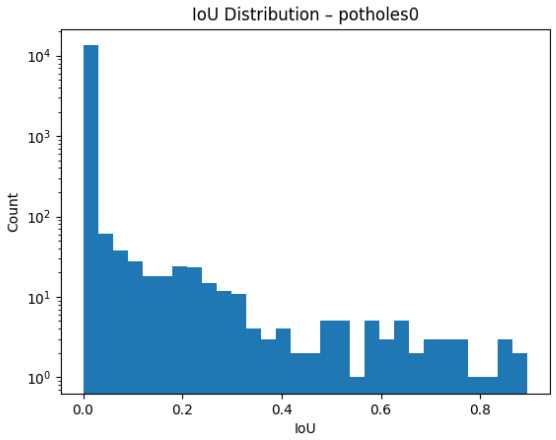

Figure 2. Histogram of IoU log-y values between region proposals and ground-truth.

- batch normalization,
- ReLU activation,
- 2×2 max pooling.

Channel depth increases progressively ($16 \to 32 \to 64 \to 128$), while spatial resolution is reduced by pooling from 128×128 down to 8×8. Features are flattened and passed through a two-layer MLP head: a hidden layer of size 128 with ReLU, and a final linear layer producing logits for two classes. This design is intentionally small to reduce overfitting and keep training fast while still capturing local texture/shape cues typical of potholes.

3.2. Dataloader and class-imbalance handling

A custom PyTorch dataset (`ProposalDataset`) was created to load proposal patches and labels stored as `.npz` files (one per image). Each file contains:

- `boxes`: proposal coordinates $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$,
- `labels`: binary labels (1 pothole, 0 background).

For each image, all positive proposals are always retained. Negative proposals are downsampled to limit imbalance: for images containing positives, a maximum of `max_neg_per_pos`×(#positives) negatives are randomly selected (default 3:1 ratio). If an image has no potholes, up to 50 background proposals are kept to still expose the model to diverse negatives.

During `__getitem__`, the dataset:

1. loads the original image via `find_image_path`,
2. crops the proposal region,
3. handles degenerate boxes by substituting a zero patch,
4. resizes to 128×128, normalizes to $[0, 1]$, and converts to CHW tensor format.

Two `DataLoaders` were then instantiated: shuffled mini-batches for training and deterministic batches for validation.

3.3. Finetuning / training procedure

Training is performed in `train_detector`. The official training split is loaded, then internally divided into an 80/20 train/validation split to monitor generalization.
Because class imbalance still exists after downsampling, weighted cross-entropy is used:

$$\mathcal{L} = -w_y \log p_y,$$

with weights $w_0 = 1.0$ (background) and $w_1 = 5.0$ (pothole), encouraging the network to prioritize recall on the rarer class. Optimization uses Adam with learning rate 10^{-3}. An early-stopping mechanism is implemented with patience 10 epochs. If validation accuracy does not improve for 10 consecutive epochs, training stops to prevent overfitting.

Table 1. Training Hyperparameters

Parameter	Value
Optimizer	Adam
Learning Rate	10^{-3}
Batch Size	64
Input Size	128×128
Loss Function	Weighted Cross-Entropy ($w_{pos} = 5.0$)
Epochs	50 (with Early Stopping)

3.4. Validation classification accuracy

After each epoch, the model is evaluated on the validation proposals in `eval` mode with gradient computation disabled. Classification accuracy is computed as:

$$\text{Acc} = \frac{\#\text{correct predictions}}{\#\text{validation samples}}.$$

This metric measures proposal-level classification performance and is distinct from full detection evaluation (which would require non-max suppression and mAP over images). The best observed validation accuracy is tracked for early stopping, and final epoch statistics are printed.
Overall, the delivered code builds a balanced proposal dataset, trains a small CNN with imbalance-aware loss, and reports proposal classification accuracy on held-out validation data.

4. Part 3 - Testing and Object Detector

4.1. Non-Maximum Suppression (NMS)

A significant challenge with Selective Search is its tendency to generate multiple overlapping bounding boxes for a single object. To resolve this redundancy and ensure that each pothole is represented by a single detection, we implemented Non-Maximum Suppression (NMS).

The NMS algorithm first sorts all detected boxes by their confidence scores in descending order. It selects the box with the highest score as a final detection and then computes the Intersection over Union (IoU) between this selected box and all remaining lower-scoring boxes. If the IoU exceeds a strict threshold of 0.3, the lower-scoring box is considered a duplicate and is suppressed. This process iterates until all candidate boxes have been either selected or discarded, effectively removing cluster detections around the same ground-truth object.

4.2. Evaluation Methodology

The performance of the object detector is quantified using the Average Precision (AP) metric. This metric summarizes the trade-off between precision and recall at different confidence thresholds. A predicted bounding box is considered a True Positive (TP) if it overlaps with a ground-truth annotation with an IoU of at least 0.5. Conversely, if the IoU is below this threshold, or if the ground-truth object has already been matched to a higher-scoring detection, the prediction is counted as a False Positive (FP). The final AP score is calculated by integrating the area under the interpolated Precision-Recall curve.

4.3. Results and Discussion

The complete object detection pipeline was evaluated on the held-out test split, which consists of 133 images. The system generated a total of 2512 detections after NMS and achieved an Average Precision (AP) of 0.206.

Figure 3 illustrates a qualitative result from the test set. The detector successfully identified the two potholes in the upper region of the image (red boxes overlapping green ground truth). However, the visualization also highlights specific limitations contributing to the low AP score. First, the model produced a False Positive in the bottom left corner, detecting a patch of textured road surface as a pothole. Second, the bounding box localization is imprecise, as the central detection is significantly larger than the ground truth, likely due to the inability of Selective Search to propose a tighter region. These issues indicate that while the SimpleCNN learns texture features, it struggles to differentiate similar road irregularities and relies heavily on the quality of the unsupervised proposals.

5. Conclusion

In this project, we successfully developed and evaluated a complete object detection pipeline for potholes in the wild. The system integrates a region proposal mechanism using Selective Search, a custom Convolutional Neural Network for classification, and Non-Maximum Suppression for post-processing. With an Average Precision of 0.206, the model demonstrates the feasibility of automated road damage detection. However, the results suggest that the current proposal-based approach has limitations regarding localization precision and false positive rates. Future work to improve robustness for real-world deployment would likely require end-to-end deep learning architectures, such as Faster R-CNN or YOLO, which can learn to optimize region proposals directly from the data.

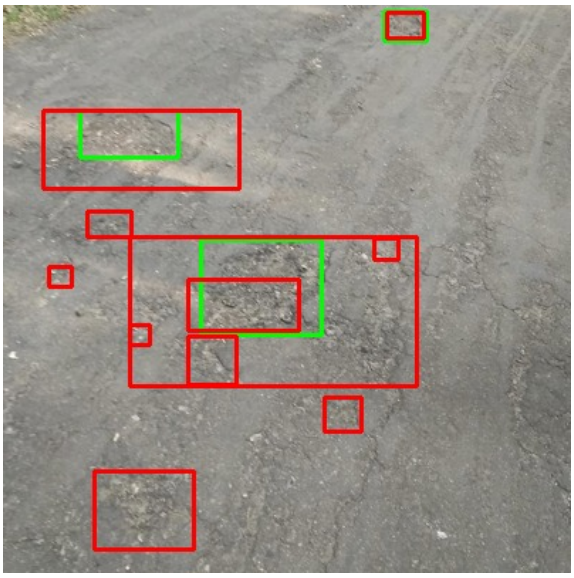

Figure 3. Qualitative detection result on test image

6. Declaration of the Use of AI Tools

We used AI tools to support our work in this project. Firstly, it was applied in the debugging process, where the tool assisted in identifying potential causes of errors and clarifying why certain parts of the code did not function as intended. This contributed to a more efficient resolution of technical issues and a deeper understanding of the underlying problems. In addition, they were used in the refinement of our written report, particularly in rephrasing and polishing the text to improve clarity, precision, and overall readability. In this way, the tool served as a supplementary aid that improved both the technical and the communicative aspects of our work.